



SOUND ANALYTICS TEAM

Isaac Gutiérrez Ortiz

Dayanna Leyton Bustos

María Andrea Marín Velasco

Juan Diego Rodríguez Delgado

Laura Jimena Bejarano Arias

Cristian Andrés Ortiz Camargo

Universidad Nacional de Colombia

2015734: Programación de Computadores

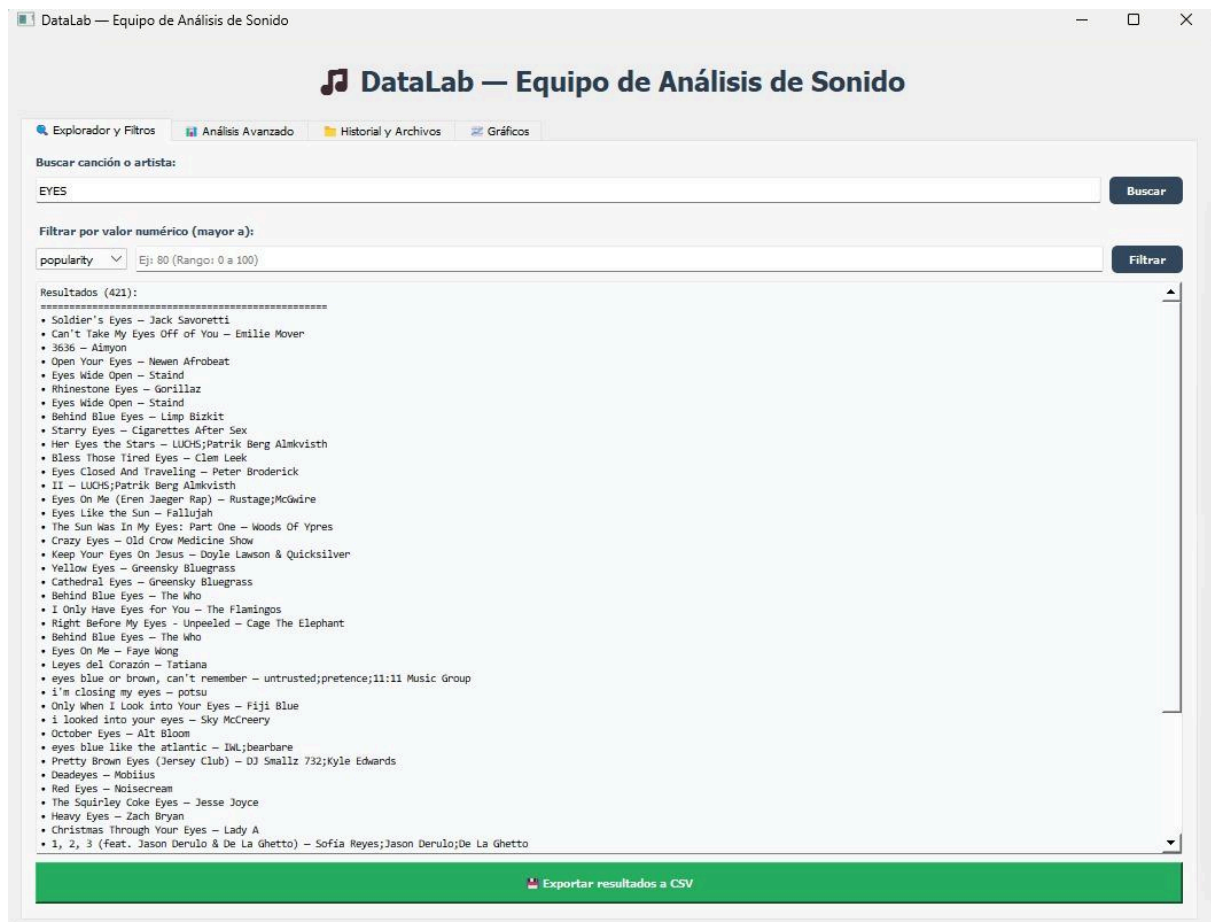
David Alberto Herrera Álvarez

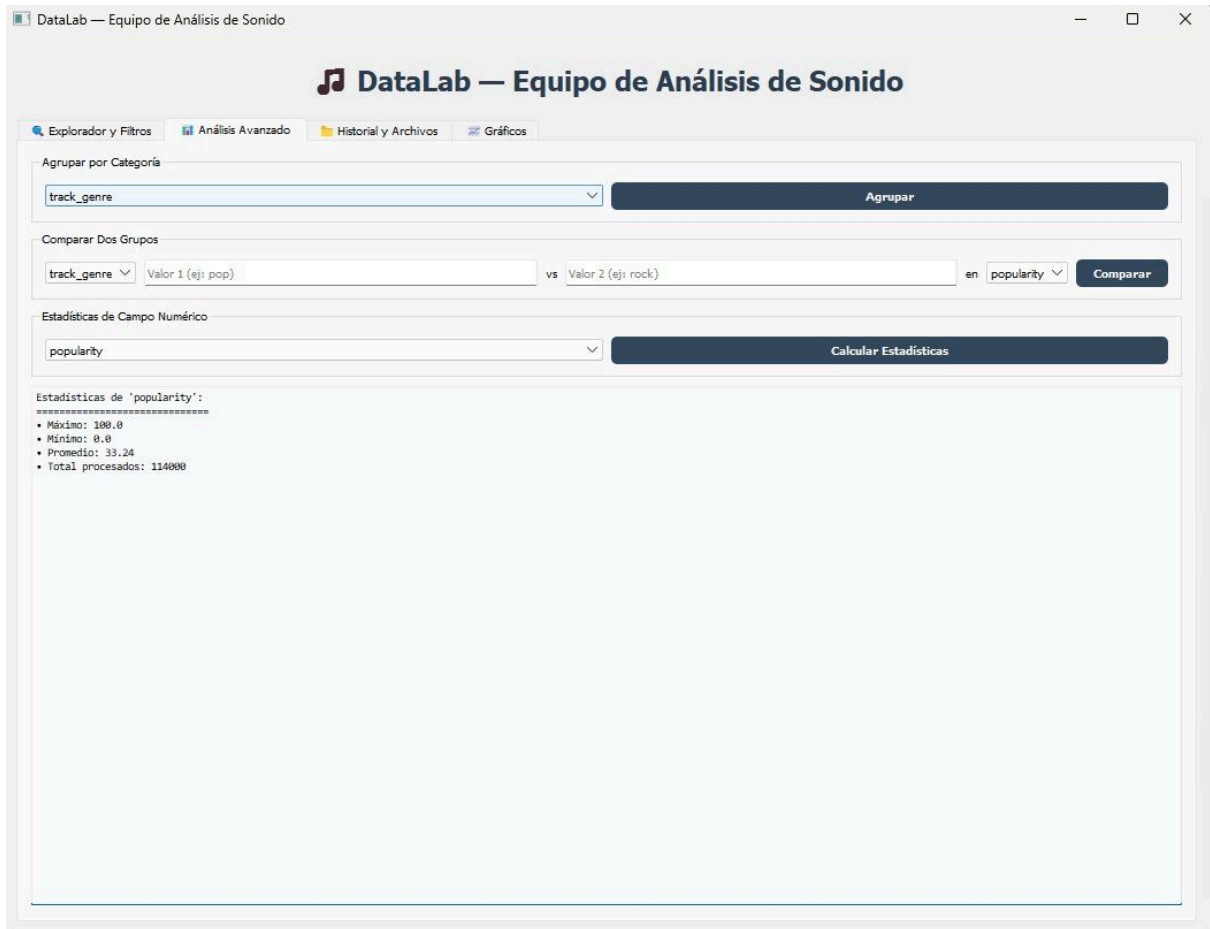
31 de mayo de 2026



1. Capturas del DataLab

- Ventana principal-panel de búsqueda y filtro







DataLab — Equipo de Análisis de Sonido

Explorador y Filtros

Análisis Avanzado

Historial y Archivos

Gráficos

Cargar y Ver Historial de Consultas

Cargar resultados previos (CSV):

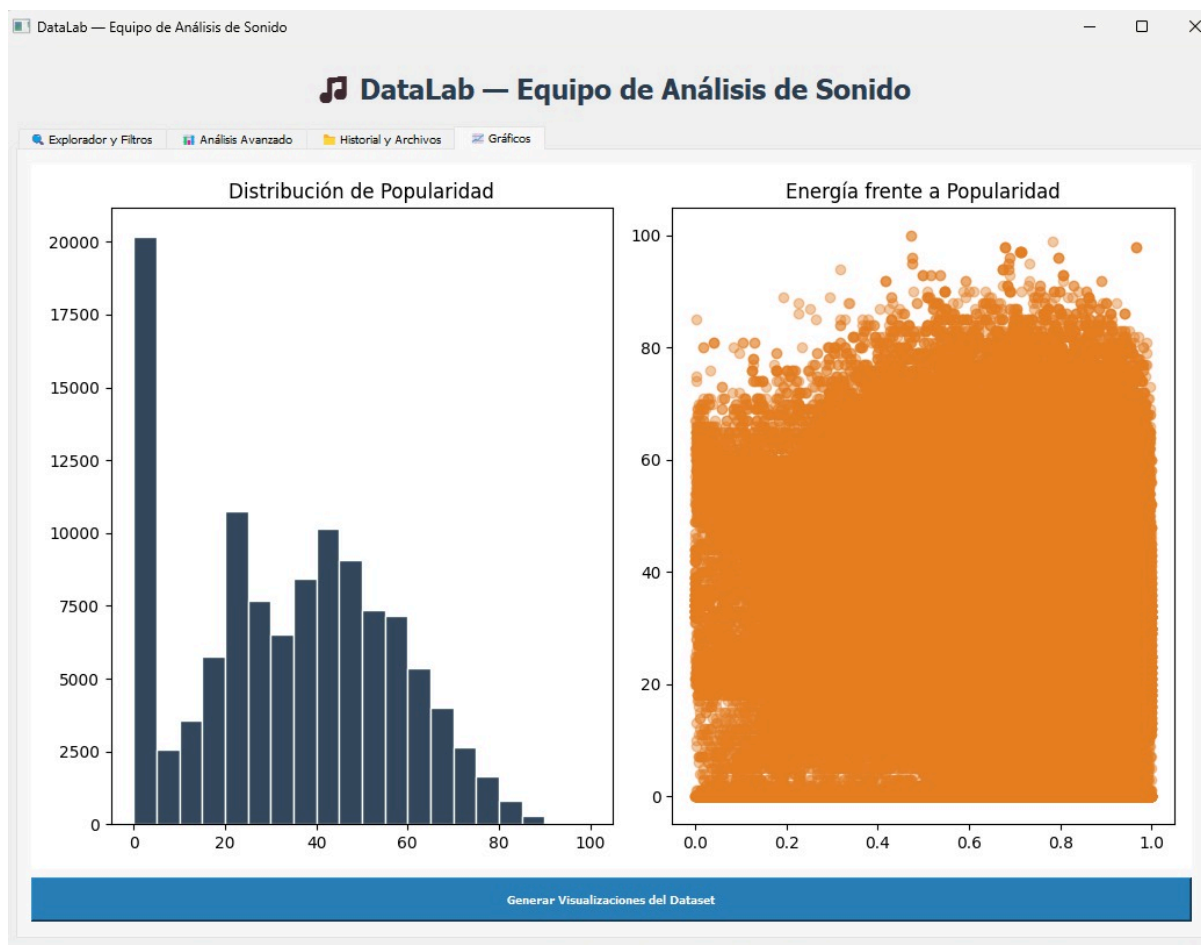
Ej: resultados_exportados.csv

Cargar Archivo

=== HISTORIAL DE CONSULTAS ===

2026-05-31 22:00,Búsqueda: 'eyes',421 resultados
2026-05-31 22:00,Búsqueda: 'eyes',421 resultados
2026-05-31 22:00,Filtro: popularity > 3.0,94219 resultados
2026-05-31 22:01,Filtro: popularity > 3.0,94219 resultados
2026-05-31 22:03,Generación de visualizaciones gráficas,0 resultados
2026-05-31 22:03,Filtro: energy > 0.2,106126 resultados
2026-05-31 22:03,Generación de visualizaciones gráficas,0 resultados
2026-05-31 22:03,Generación de visualizaciones gráficas,0 resultados
2026-05-31 22:07,Generación de visualizaciones gráficas,0 resultados
2026-05-31 22:07,Filtro: valence > 0.6,38666 resultados
2026-05-31 22:08,Generación de visualizaciones gráficas,0 resultados
2026-05-31 22:09,Búsqueda: 'eyes',421 resultados
2026-05-31 22:18,Filtro: energy > 0.8,37880 resultados
2026-05-31 22:22,Búsqueda: 'eyes',421 resultados
2026-05-31 22:25,Agrupación por track_genre,114 resultados
2026-05-31 22:25,Agrupación por explicit,2 resultados
2026-05-31 22:25,Agrupación por time_signature,5 resultados
2026-05-31 22:25,Agrupación por mode,2 resultados
2026-05-31 22:26,Agrupación por key,12 resultados
2026-05-31 22:26,Comparación: pop vs rock en popularity,2 resultados
2026-05-31 22:26,Comparación: pop vs rock en energy,2 resultados
2026-05-31 22:26,Comparación: pop vs rock en tempo,2 resultados
2026-05-31 22:26,Comparación: pop vs rock en tempo,2 resultados
2026-05-31 22:26,Estadísticas de energy,1 resultados
2026-05-31 22:26,Estadísticas de tempo,1 resultados
2026-05-31 22:27,Estadísticas de duration_ms,1 resultados
2026-05-31 22:27,Estadísticas de popularity,1 resultados
2026-05-31 22:27,Búsqueda: 'eyes',421 resultados
2026-05-31 22:29,Búsqueda: 'EYES',421 resultados
2026-05-31 22:29,Filtro: energy > 0.5,82059 resultados
2026-05-31 22:29,Filtro: energy > 0.5,82059 resultados
2026-05-31 22:29,Agrupación por track_genre,114 resultados
2026-05-31 22:29,Estadísticas de popularity,1 resultados
2026-05-31 22:29,Estadísticas de popularity,1 resultados
2026-05-31 22:34,Filtro: duration_ms > 0.5,113999 resultados
2026-05-31 22:36,Agrupación por track_genre,114 resultados
2026-05-31 22:36,Estadísticas de energy,1 resultados
2026-05-31 22:36,Estadísticas de popularity,1 resultados
2026-05-31 22:36,Búsqueda: 'EYES',421 resultados

- Gráficos embebidos en la ventana



2. Descripción del código

Este programa funciona como un analizador interactivo de datos de canciones de Spotify almacenados en archivos CSV. Al iniciar, carga un conjunto de datos y muestra un menú con diferentes opciones que permiten buscar canciones por palabras clave, calcular estadísticas de campos numéricos, filtrar registros según un valor mínimo, agrupar canciones por categorías, comparar grupos de canciones utilizando promedios de una característica numérica, consultar el historial de operaciones realizadas y cambiar el archivo de datos con el que se está trabajando. El programa utiliza funciones de los módulos `archivos` y `analisis` para cargar, procesar y guardar información, registra las consultas efectuadas en un historial y ofrece la posibilidad de guardar los resultados obtenidos en nuevos archivos CSV. Todo el funcionamiento se desarrolla dentro de un ciclo que mantiene activo el menú hasta que el usuario selecciona la opción de salir.

Además, el sistema está diseñado para facilitar el análisis y la exploración de los datos de manera organizada, lo cual permite al usuario obtener información relevante sobre



las canciones almacenadas en la base de datos. Gracias a su estructura modular, las diferentes funcionalidades se ejecutan de forma eficiente y pueden ampliarse fácilmente para incorporar nuevas opciones de análisis o procesamiento de datos en futuras versiones del programa.

3. Instrucciones de uso

1. Antes de ejecutar el programa, se debe tener instalado:

- Python 3
- Pandas
- PyQt5

Para instalar las librerías necesarias, usar: `pip install pandas PyQt5`

2. **Descargar los archivos del repositorio:** es imprescindible asegurarse de descargar los archivos del repositorio y organizarlos en una misma carpeta:

- a. `main.py` (código principal)
- b. `analisis.py` (Lógica algorítmica)
- c. `archivos.py` (Gestor de archivos)
- d. `spotify completo.csv` (Dataset principal con más de 200 filas)
- e. `spotify pequeño.csv` (Subconjunto de pruebas de 50 filas)

Se recomienda usar las primeras 500 filas del dataset completo puesto que el archivo contiene más de 100.00 filas y puede saturarse.

3. **Preparación:** Abra su consola de comandos, diríjase a la carpeta del proyecto y ejecute el archivo Python '[main.Py](#)'. Aparecerá el menú principal con 8 opciones numéricas, para ejecutar cualquiera de ellas debe digitar el número correspondiente.
4. **Opción 1 (Búsqueda General):** Permite escribir cualquier palabra y el sistema buscará coincidencias y arrojará una lista con los resultados en pantalla.
5. **Opción 2 (Métricas Estadísticas):** al introducir el nombre exacto de una columna numérica (ej: popularity, energy, tempo). La consola calculará y mostrará el valor máximo, mínimo y el promedio de los registros cargados.
6. **Opción 3 (Filtrado Numérico):** Se usa para aislar datos, para ello debe ingresar un campo numérico, un operador (> o <) y un valor límite. El sistema imprimirá los registros que superen o queden por debajo de dicho umbral.
7. **Opción 4 (Distribución/Ranking):** Agrupa y cuenta cuántas canciones pertenecen a cada categoría dentro de un campo de texto de forma ordenada (de mayor a menor) generando un histograma visual básico en consola.
8. **Opción 5 (Comparación):** Elija un campo de texto (ej: track_genre), escriba dos grupos diferentes (ej: pop y rock) y una métrica numérica. El programa validará su existencia con conjuntos (sets) y contrastará sus promedios y límites.



Guardar resultados - persistencia Cuando realice una búsqueda (Opción 1) o un filtrado (Opción 3), al terminar de listar las canciones, el sistema le preguntará: ¿Desea guardar estos resultados en un nuevo archivo CSV? (s/n). Presione (s) si desea guardarlos y asigne un nombre al nuevo archivo. Se creará un archivo legible separado por comas de forma automática en el directorio.

9. **Opción 6 (Registro automático):** Cada acción realizada por usuario queda registrada automáticamente. Esta opción le permitirá consultar el historial acumulado hasta el momento, para lo cual imprimirá una tabla estructurada directamente desde el archivo autogenerado con el nombre de su elección, detallando: la fecha y hora exacta, la operación realizada, los criterios ingresados y el volumen de filas afectadas.
9. **Opción 7 (Recuperar Archivos Guardados):** Si cerró el programa y desea trabajar únicamente con el subconjunto de datos que guardó en un archivo en el Paso 3 sin procesar la base completa, esta opción le permitirá hacerlo, solo debe ingresar el nombre del archivo guardado. El sistema cambiará la fuente de datos en memoria y todas las búsquedas o estadísticas posteriores se calcularán sólo sobre ese archivo.
10. **Opción 8 (Cierre Seguro):** Seleccione esta opción para finalizar la sesión y asegurar el correcto cierre.

4. Funcionamiento del código

El código está organizado de forma modular y se divide en varias funciones que facilitan su mantenimiento y comprensión. Define funciones auxiliares para mostrar la información de una canción (`imprimir_cancion`), guardar resultados (`preguntar_guardado`) y presentar el menú principal (`mostrar_menu`). La función `main()` coordina toda la ejecución del programa: carga el dataset inicial, obtiene los encabezados de las columnas y ejecuta un ciclo `while` que muestra el menú y procesa la opción elegida por el usuario mediante una estructura `if-elif`. Dependiendo de la opción seleccionada, se llaman funciones de los módulos `archivos` y `analisis` para realizar búsquedas, cálculos estadísticos, filtrados, agrupaciones y comparaciones. Además, el programa incorpora validación básica de entradas mediante bloques `try-except`, registra las operaciones realizadas en un historial y permite cambiar dinámicamente el archivo de datos cargado. Para la visualización de datos, utiliza la biblioteca Matplotlib para generar gráficos que representan de forma visual los resultados de los análisis realizados, facilitando la interpretación de tendencias y comparaciones dentro del conjunto de datos. Finalmente, la instrucción `if __name__ == "__main__":` asegura que la función principal se ejecute únicamente cuando el archivo se utilice como programa principal y no cuando sea importado desde otro módulo, una de esas pequeñas decisiones de Python que evitan dolores de cabeza futuros. Asimismo, la estructura modular del programa permite que cada componente desempeñe una función específica dentro del procesamiento de los datos, favoreciendo una mejor organización del código y facilitando su mantenimiento. La interacción entre los módulos de análisis y gestión de archivos permite ejecutar las diferentes operaciones de manera eficiente, mientras que el uso de funciones



independientes mejora la legibilidad del programa y simplifica la incorporación de nuevas funcionalidades o ajustes en futuras versiones.

Instrucciones para la creación de gráficos

Este proyecto utiliza la biblioteca Matplotlib para crear gráficos y pandas para procesar los datos del archivo ``spotify_completo.csv``.

Primero, el programa carga el archivo CSV con pandas. Luego, agrupa los datos según la información que se va a presentar. En este ejemplo se crean dos gráficos principales:

1. Gráfico de barras: Número de canciones por género musical

Para este gráfico, el programa extrae la columna correspondiente al género musical y cuenta el número de canciones en cada categoría. Esto se logra mediante la función ``value_counts()``, que recupera el número de canciones para cada género musical.

Luego, estos datos se presentan como un gráfico de barras. El eje X representa el género musical y el eje Y representa el número de canciones en cada género.

Este gráfico ayuda a determinar qué géneros musicales tienen la mayor proporción en el conjunto de datos.

2. Gráfico de líneas: Popularidad promedio por género

En el segundo gráfico, el programa agrupa las canciones por género y calcula la popularidad promedio de cada grupo. Esto se logra utilizando las funciones ``groupby()`` y ``mean()`` de la biblioteca pandas.

Los resultados se muestran como un gráfico de líneas. El eje X representa el género musical y el eje Y la popularidad promedio.

Este gráfico compara y analiza visualmente los géneros con la mayor popularidad promedio en el archivo.

Integración del gráfico en una ventana

En lugar de mostrar el gráfico en una ventana aparte con ``plt.show()``, se integra directamente en la interfaz gráfica creada con PyQt5.

Esto se logra utilizando ``FigureCanvasQTAgg``, que convierte el gráfico de Matplotlib en un control compatible con PyQt5. Este control se agrega al diseño de la ventana con ``layout.addWidget()``.

De esta forma, el gráfico se integra en la aplicación, pasando a formar parte de la interfaz principal del sistema.